I.N.T.A.C.T. — Integrity of Neural Transformers Across Compression Techniques

Project Outline

What are we doing?

AI models get compressed before they ship. Every number inside the model is rounded to something coarser so the file is small enough to run on a phone. Companies check the compressed version by confirming its benchmark scores held up.

Separately, researchers have traced which specific parts of a model produce specific behaviours. They proved it causally: switch the part off, the behaviour breaks.

Nobody has checked whether those traced parts still do their job after the model is compressed. A benchmark average can stay flat while a different part of the model quietly takes over the work. That substitution is invisible to every standard evaluation.

We measure it.

What is already known? What are we adding?

In June 2026, Duan published "Perplexity Can Miss SAE Feature Damage Under Quantization" (arXiv 2606.03002). Using a sparse autoencoder as a fixed measuring stick, that work showed that on Gemma-2-2B, INT7 quantization improved perplexity while degrading 18.7% of active features. At INT6, only 51.3% of features survived.

So the headline finding, that compression changes a model's insides while scores stay flat, is established.

What the paper did not do:

1. **It measured correlation between feature activations before and after rounding**. That is a geometric claim about where a vector points, not a *causal claim about whether the mechanism still works*. A feature can drift and still function, or hold steady and stop functioning.
2. It **studied sparse autoencoder features**, which are anonymous dictionary entries. *We study circuits whose components have known job titles,* so we can ask which role in an algorithm fails first.
3. It did not test **whether structural redundancy protects a circuit**.

The paper names causal validation as its own next step. That is our project. Building visibly on a recent result is a stronger position than claiming an empty field.

Hypothesis

H1: Decoupling: There exists a precision where the mechanism is measurably broken and the benchmark is not

There are three levels at which you can measure a quantized model:

1. General task loss, such as perplexity on a text benchmark.
2. Task behavior, such as whether the model still gets IOI sentences right.
3. Circuit-level causal effect, such as how much patching the circuit's heads still moves the output compared with the full-precision model.

H2: structure predicts fragility. The tightly composed induction circuit (2 heads, no known redundancy) degrades before the IOI circuit (26 heads, 7 functional classes, documented backup components).

H3: Damage comes from quantization error in the circuit's own weight matrices, not from accumulated noise elsewhere in the network.

Quantization rounds every weight in the model, not just the circuit's. Each rounded weight matrix produces slightly wrong outputs, and those small errors don't stay put. They get added into the residual stream and carried forward to every later layer.

That gives two routes to a broken circuit:

1. **Local damage:** The circuit's own weights (e.g. the Q, K, V, O matrices of the two induction heads) are rounded, so the heads compute a slightly wrong function. The error is inside the mechanism.
2. **Inherited damage:** The circuit's weights are fine, but earlier layers were rounded. The residual stream arriving at the circuit is already slightly off, so the heads get perturbed inputs. A previous-token head upstream, an MLP, or an embedding could all contribute. The mechanism is intact, but it's being fed corrupted data.

Some Vocab

Term	Meaning
Parameter / weight	One number stored in the model. GPT-2 small has 124 million.
Attention head	One small component. GPT-2 small has 144, in 12 layers of 12. Written as L9H9 or 9.9, meaning layer 9, head 9.
Residual stream	The shared workspace every component reads from and adds to. Nothing overwrites, everything accumulates.
Circuit	A small group of heads that together implement one identifiable behaviour.
Quantization	Rounding every weight to a coarser grid. INT8 gives 256 possible values, INT4 gives 16.
Fake quant (Q-DQ)	Quantize then immediately convert back to float. Numerically identical to real low-bit weights, but keeps our tooling working.
Ablation	Switch a component off and see what breaks. Crude.
Activation patching	Copy one cached activation from a clean run into a corrupted run, measure how much the output recovers. This is our core tool.
DLA	Direct logit attribution. How much one head pushed the answer toward a specific token.
FP16 baseline	The uncompressed reference. Every number we report is normalized against it.

Two Circuits We Are Studying

Induction heads: Behaviour: given [A][B] ... [A], predict [B]. Implemented by two heads working together. An early-layer previous-token head writes "the token before me" into the residual stream, and a later-layer induction head reads that to find where the current token appeared before, then copies what followed it. Tight, no known backups. We predict this one breaks first.

The IOI circuit: Behaviour: "Sarah and Tom went to the shop. Sarah gave a drink to ___" returns Tom. Implemented by 26 heads in 7 classes: duplicate token heads, previous token heads, induction heads, S-inhibition heads, name mover heads, negative name mover heads, and backup name mover heads.

The backup name movers matter more than anything else in this document. They are mostly idle, but when the primary name movers are ablated, they increase their contribution and partially restore the behaviour. That is published, direct evidence that a transformer contains latent redundancy which activates on damage. It is the mechanism by which a circuit could break while the benchmark stays flat

---

What We Need To Build

1. tasks.py — makes the test questions.
Generates hundreds of “Sarah gave a drink to ___” sentences plus the repeated-token sequences, from templates. Exists because we ask the model the same questions at all 42 settings, so they have to be generated from a seed rather than written by hand, or we can’t reproduce anything.

2. circuits.py — the watch list.
Just the list of which heads belong to which circuit, and which job each one does. Exists so there’s one place that answers “which parts matter,” instead of head numbers hardcoded across five other files where they’ll drift out of sync.

3. quantize.py — the rounder.
Takes the model and a bit-width, blurs every number to that coarseness. Exists because this is the thing we’re actually studying. It’s our one independent variable, the only thing we change between runs.

4. metrics.py — the simple measurements.
Did the model get the right answer, how confident was it, is each head still looking at the same word. Exists to give us the outside view, the one a company would check before shipping.

5. patching.py — the causal measurement.
Switches one part’s input for another and measures how much the answer moves. Exists to give us the inside view: not “does this part look normal” but “is this part still causing the answer.” This file is the entire reason our project is different from the prior work. Also the hardest to get right.

6. run_sweep.py — the loop.
Runs all 42 combinations and writes one row per measurement. Exists so every condition is run identically with a fresh model, and nothing is ever run by hand in a notebook where the settings are forgotten by Tuesday.

7. analyze.py — turns rows into curves.
Reads the spreadsheet, normalizes, adds error bars, finds where each line drops below 90%. Exists as a separate file from collection so we can redo the analysis a dozen times without re-running the sweep, and so the stats are reproducible from the raw data.

Therefore In A Perfect World:

1. **Build the test questions and the watch list**. The sentences we test on, plus the list of which parts we're tracking. Both are mostly data entries. The one trap is that each test sentence needs a near-identical twin with just the name swapped, and if anything else differs between the two, every measurement later is quietly wrong.

2. **Build the rounder, then leave it alone**. The function that blurs the model's numbers to a chosen coarseness. Easiest piece in the project, maybe 80 lines. Build it early because it's easy, but do not use it yet.

3. **Build the measuring tools**. Four measurements: did the model get the answer right, is each watched part still causing that answer, is it still looking at the same place in the sentence, and does the circuit still work with everything else switched off. The second one is the patching code and it's the hardest thing we write.

4. **Prove the measurements are right on the normal model**. Run everything on uncompressed GPT-2 and check our numbers match the published paper's numbers. This is the gate. If they don't match, our measuring tools are broken, and every result after this is garbage that looks fine. Nothing gets compressed until this passes. It's tempting to skip ahead and we won't.

5. **Check the pipeline doesn't invent drift**. Run the uncompressed model against itself through the whole thing. It must report exactly zero change. If it reports anything else, the pipeline is manufacturing results on its own.

6. **Write down what we expect, and date it**. Before anyone sees a single compressed result. If we're right, the evidence is much stronger because we called it in advance. If we're wrong, the surprise is the interesting part and we can prove we didn't invent the story afterward.

7. **Run the sweep**. 42 combinations, fresh model each time, one row per measurement into a spreadsheet. Conceptually trivial. It will still break repeatedly because something upstream hands back an unexpected shape.

8. **Turn the spreadsheet into curves**. Normalize everything against the uncompressed model so all lines start at 1.0, add error bars, and find where each line crosses below 90%. The gap between where the test score breaks and where the circuit breaks is the entire finding.

9. **Try to break our own result**. Different layers, different thresholds, different random seeds, second model if there's time. If the finding only holds under one exact setup, we found a bug, not a result.